FACULTAD DE INFORMÁTICA

UNIVERSIDAD NACIONAL DE LA PLATA

Taller de Proyecto II

Implementación FABRIK

Grupo J1
Tarifa, Armando Ezequiel[1] (02893/4)
Guitera Woida, Emma[1] (03352/2)
Terruzzi, Ana Justina[1] (02557/9)

17/10/2026

[1]{ezequiel.tarifa.2002,emma.guitera.woida,justina.terruzzi}@gmail.com

El grupo J1 va a controlar el brazo robótico de 5 grados de libertad con una versión de FABRIK adaptada a su estructura. En lugar de la cadena 3D con rótulas libres del modelo general presentado en la investigación de modelos, se adapta el método para bisagras del paper original, que resuelve el problema dentro del plano vertical del brazo. Dentro de ese plano el codo puede terminar doblado hacia el lado equivocado, y ese problema se resuelve con la técnica *pole* de Blender y Maya. El usuario solo indica el punto al que tiene que llegar la pinza. FABRIK calcula la base, el hombro y el codo, y la muñeca y la pinza se eligen con una búsqueda que empieza en su posición neutra. Los límites articulares son los que permite el firmware, se aplican en cada paso y se expresan en grados del servo.

Índice

1. Introducción 3
2. El brazo como cadena cinemática 3
   2.1. Estructura física 3
   2.2. Geometría del modelo 3
   2.3. El brazo se mueve en un plano vertical 4
3. FABRIK según el paper 5
   3.1. La idea del algoritmo 5
   3.2. Restricciones articulares 5
   3.3. Resultados reportados en el paper 6
   3.4. Limitación reportada 6
4. De rótulas libres a bisagras 6
   4.1. El modelo general de la investigación 6
   4.2. Aplicación directa a nuestro brazo 6
   4.3. Por qué se eligió el método de bisagras 7
5. FABRIK en el plano del brazo 8
   5.1. Paso 0: alcanzabilidad 8
   5.2. Paso 1: el plano del brazo 8
   5.3. La cadena virtual hombro, codo, $\boldsymbol{p}_5$ 9
   5.4. Paso 2: iteración dentro del plano 9
   5.5. Paso 3: recuperación de los ángulos 9
   5.6. Ejemplo completo 9
6. El problema del codo abajo 10
   6.1. Dos maneras de llegar al mismo punto 10
   6.2. Qué hace FABRIK sin una regla para el codo 11
   6.3. Ajustar los ángulos al final no resuelve el problema 11
   6.4. Limitar en cada paso tampoco alcanza 11
   6.5. Magnitud del problema 11
7. La solución: la técnica *pole* de Blender y Maya 13
   7.1. El problema en animación 13

7.2. Cómo se traduce a nuestro brazo ... 13
7.3. Detalles de la regla ... 14
8. Restricciones articulares y validación ... 15
8.1. Restricciones en cada paso ... 15
8.2. Criterios de corte ... 15
8.3. Validación final, silueta y colisiones ... 15
9. Qué calcula FABRIK y cómo se eligen la muñeca y la pinza ... 17
10. Convención de ángulos y límites ... 17
11. Versionado del modelo ... 18
12. Plan de implementación y pruebas ... 19
12.1. Simulación en Python ... 19
12.2. Pasaje a C (ESP32) ... 19
12.3. Pendientes ... 19
12.4. Mejora a futuro: la pinza dentro de FABRIK ... 19
13. Conclusiones ... 21
Referencias ... 22

1. Introducción

El grupo J1 implementa para el brazo robótico una versión de FABRIK (*Forward And Backward Reaching Inverse Kinematics*) adaptada a su base giratoria y a sus bisagras paralelas. Primero se programa en la simulación en Python (`code/simulacion/`), donde puede visualizarse y probarse sin el brazo físico, y una vez validada se porta a C para el ESP32.

Este informe complementa al documento *Investigación de Modelos*, donde se justifica la elección de FABRIK frente al Jacobiano transpuesto, DLS y CCD y se presenta el modelo general del algoritmo. Aquí se explica cómo se adapta ese modelo a nuestro brazo y qué se decidió frente a cada problema que apareció al probarlo.

Como referencia principal se toma el paper original de FABRIK [1]. Además, se consultaron:

- la documentación de Blender y Maya sobre la técnica del *pole* [2], [3], [4];
- el informe final del grupo J5 (2025) [5].

Los valores numéricos salen de la simulación en Python, que usa la cadena cinemática de `configuracion.c`, replicada en `configuracion.py`, con las medidas del brazo armado (Sección 2.2). Las figuras del brazo se dibujaron con las coordenadas que calcula la cinemática directa del modelo.

2. El brazo como cadena cinemática

2.1. Estructura física

El brazo está formado por cinco servomotores SG90 conectados en serie. La Figura 1 resume la cadena completa.

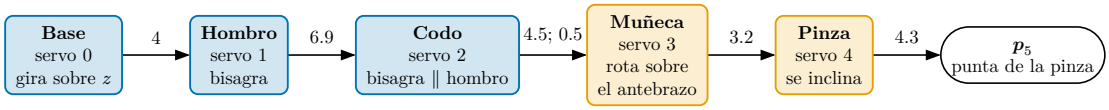


Figura 1. Cadena cinemática del brazo. Los números sobre las flechas son las distancias en cm entre articulaciones, medidas en el brazo armado, salvo el desvío de 0.5 cm del antebrazo, que es el de `configuracion.c`. En azul, los servos que calcula FABRIK; en naranja, la muñeca y la pinza, que elige la búsqueda de la Sección 9.

2.2. Geometría del modelo

La cinemática directa heredada del grupo J5 construye cada eslabón con matrices homogéneas, que combinan una rotación fija de montaje R_{fix}, la rotación del servo alrededor de su eje z local y la traslación del eslabón:

$$M^{(i)}_{\text{eslabón}} = M_{\text{acum}} \cdot R^{(i)}_{\text{fix}} \cdot R_z(\theta_i), \quad M_{\text{acum}} \leftarrow M^{(i)}_{\text{eslabón}} \cdot T(\text{SALTO}_i)$$

Con las medidas del brazo, en la *pose cero* (todos los ángulos del modelo en 0°) las articulaciones quedan en las posiciones de la Tabla 1, que se dibujan en la Figura 2. El *home* del firmware es otra pose. Lleva los servos a 400, 1400, 1200, 960 y 400 µs, que en ángulos del modelo son $(0°, 140°, -114.6°, 4.5°, 0°)$, con el brazo inclinado hacia atrás y el codo doblado. Es la pose en la que arranca el brazo y la misma que muestra el gemelo digital en la Figura 5 del informe final del grupo J5 [5].

Tabla 1. Posición y eje de giro de cada articulación en la pose cero, calculados con la cinemática directa del modelo.

Punto	Articulación	Posición en la pose cero (cm)	Eje de giro en la pose cero
$\boldsymbol{p}_0$	Base (servo 0)	$(0, 0, 0)$	z (vertical)
$\boldsymbol{p}_1$	Hombro (servo 1)	$(0, 0, 4)$	$-y$ (horizontal)
$\boldsymbol{p}_2$	Codo (servo 2)	$(6.9, 0, 4)$	$-y$ (paralelo al hombro)
$\boldsymbol{p}_3$	Muñeca (servo 3)	$(11.4, 0, 4.5)$	$-x$ (a lo largo del antebrazo)
$\boldsymbol{p}_4$	Pinza (servo 4)	$(14.6, 0, 4.5)$	$-y$
$\boldsymbol{p}_5$	Punta de la pinza	$(14.6, 0, 0.2)$	—

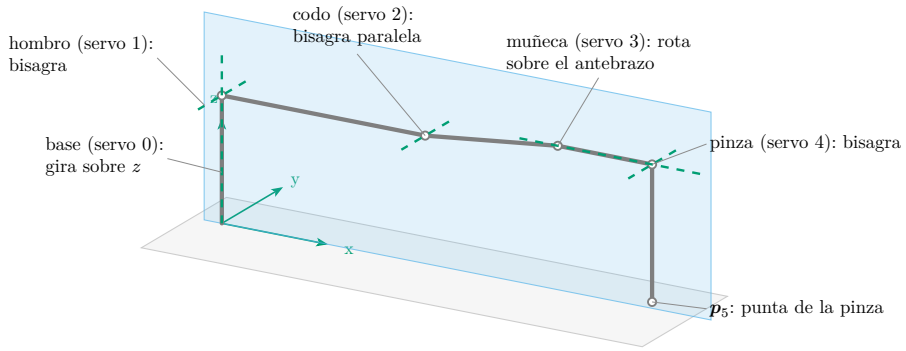


Figura 2. Brazo en la pose cero. En verde punteado se ven los ejes de giro. El de la base es vertical, los del hombro, el codo y la pinza son horizontales y paralelos, y el de la muñeca va a lo largo del antebrazo. Con la pinza en 0° la mano queda perpendicular al antebrazo, hacia abajo, y su punta queda 0.2 cm por encima del piso. La pose cero es una referencia del modelo y el brazo no la adopta. El plano celeste es el plano vertical del brazo.

Para FABRIK importan tres detalles del modelo:

1) Las medidas se tomaron sobre el brazo armado, porque las de `configuracion.c` vienen del grupo J5, cuyo informe final no trae ninguna longitud. El eje del hombro está a 4 cm de la mesa, el brazo mide 6.9 cm y el antebrazo 7.7 cm hasta el eje de la pinza: 4.5 cm hasta el centro del servo de la muñeca y 3.2 cm más hasta el eje del servo 4. La punta de la pinza está a 4.3 cm de ese eje y a 5.3 cm de la base impresa de la pinza, donde se atornilla el horn de la muñeca. De estas medidas dependen la altura del hombro, l_1, $\boldsymbol{v}$, α y d, pero no el método.
2) $\boldsymbol{p}_5$ es la punta de la pinza, a 4.3 cm de $\boldsymbol{p}_4$ a lo largo de la mano, en el plano en que gira la pinza. El servo 4 inclina la pinza y con eso mueve la punta. Con la pinza en 90° la mano queda alineada con el antebrazo y $\boldsymbol{p}_5$ cae sobre el eje de la muñeca, así que girar la muñeca no lo mueve; con la pinza inclinada, la muñeca lo saca del plano del brazo. En `configuracion.c`, `SALTO_4` llega al extremo del eje de la pinza, 2 cm hacia un costado; el grupo J5 calculaba ese punto pero no lo usaba. `configuracion.py` usa la punta medida, y `configuracion.c` se corrige en el pasaje a C, junto con las demás medidas.
3) El antebrazo tiene un pequeño desvío: de $\boldsymbol{p}_2$ a $\boldsymbol{p}_3$ hay 4.5 cm a lo largo del antebrazo y 0.5 cm hacia arriba. Los 0.5 cm son los de `configuracion.c`, porque no se midieron sobre el brazo. Este desvío y la inclinación de la pinza determinan el ángulo α, fijo para cada posición de la muñeca y la pinza, que se define en la Sección 5.3.

2.3. El brazo se mueve en un plano vertical

Como la base gira todo el brazo alrededor de z y el hombro y el codo son bisagras de ejes paralelos y horizontales, en el modelo el hombro, el codo, la muñeca y el servo de la pinza ($\boldsymbol{p}_1$ a $\boldsymbol{p}_4$) quedan siempre en un mismo plano vertical, que contiene al eje de la base y gira con ella. Solo $\boldsymbol{p}_5$ puede salir de ese plano. Con la muñeca neutra la pinza gira dentro del plano y $\boldsymbol{p}_5$ queda en él; con la muñeca girada y la pinza inclinada, queda hasta 4.3 cm hacia un costado (Tabla 5).

Por lo tanto, ubicar el efector final se puede dividir en dos problemas: elegir hacia dónde apunta el plano (la base) y resolver dentro de ese plano un brazo de dos eslabones (hombro y codo). El método de la Sección 5 se basa en esa división.

3. FABRIK según el paper

3.1. La idea del algoritmo

FABRIK resuelve la cinemática inversa trabajando con las *posiciones* de las articulaciones y no con sus ángulos [1]. Cada nueva posición se obtiene como un punto sobre una recta, a una distancia conocida. Por lo tanto, no hacen falta matrices ni derivadas.

En la notación del paper, $\boldsymbol{p}_1, ..., \boldsymbol{p}_n$ son las posiciones de las articulaciones, donde $\boldsymbol{p}_1$ es la raíz y $\boldsymbol{p}_n$ el efector final; $\boldsymbol{t}$ es el objetivo y $\boldsymbol{b}$ la posición inicial de la raíz. El objetivo no forma parte de la cadena; el algoritmo busca que $\boldsymbol{p}_n$ coincida con $\boldsymbol{t}$. Las distancias entre articulaciones son constantes:

$$d_i = \|\boldsymbol{p}_{i+1} - \boldsymbol{p}_i\|, \quad i = 1, ..., n-1$$

Antes de iterar se verifica si el objetivo está al alcance. Si $\|\boldsymbol{t} - \boldsymbol{p}_1\| > \sum d_i$, el objetivo es inalcanzable. En ese caso el paper no itera y estira la cadena en línea recta hacia el objetivo:

$$\lambda_i = \frac{d_i}{\|\boldsymbol{t} - \boldsymbol{p}_i\|}, \quad \boldsymbol{p}_{i+1} = (1 - \lambda_i)\boldsymbol{p}_i + \lambda_i \boldsymbol{t}, \quad i = 1, ..., n-1$$

Si el objetivo está al alcance, cada iteración tiene dos etapas (Algoritmo 1 del paper):

- **Forward** (de la punta a la raíz): se coloca el efector final sobre el objetivo, $\boldsymbol{p}'_n = \boldsymbol{t}$, y cada articulación se ubica sobre la recta que la une con la siguiente, a la distancia correcta:

$$\lambda_i = \frac{d_i}{\|\boldsymbol{p}'_{i+1} - \boldsymbol{p}_i\|}, \quad \boldsymbol{p}'_i = (1 - \lambda_i)\boldsymbol{p}'_{i+1} + \lambda_i \boldsymbol{p}_i, \quad i = n-1, ..., 1$$

- **Backward** (de la raíz a la punta): la raíz quedó desplazada y se la devuelve a su posición inicial, $\boldsymbol{p}''_1 = \boldsymbol{b}$; después se repite el procedimiento hacia la punta:

$$\lambda_i = \frac{d_i}{\|\boldsymbol{p}'_{i+1} - \boldsymbol{p}''_i\|}, \quad \boldsymbol{p}''_{i+1} = (1 - \lambda_i)\boldsymbol{p}''_i + \lambda_i \boldsymbol{p}'_{i+1}, \quad i = 1, ..., n-1$$

El paper plantea tres criterios de corte (sección 4):

1) la distancia entre el efector y el objetivo es menor que una tolerancia;
2) el efector deja de moverse entre una iteración y la siguiente, lo que ocurre cuando el objetivo no se puede alcanzar;
3) se superó un número máximo de iteraciones.

3.2. Restricciones articulares

En la sección 4.3 del paper se explica cómo agregar límites articulares. Como FABRIK es iterativo, la restricción se aplica *en cada paso*: cada vez que se calcula una nueva posición, se verifica si respeta los límites de la articulación y, si no los respeta, se la reubica dentro del rango permitido antes de calcular la siguiente. Esto se hace tanto en el forward como en el backward, en todas las iteraciones. Según el paper, los algoritmos de cinemática inversa suelen ser más efectivos si las restricciones se aplican en cada paso que si se aplican al final. También afirma que, si el objetivo está al alcance y existe una configuración de las articulaciones que permita llegar, FABRIK con restricciones la encuentra. En la Sección 6 se muestra que en nuestro brazo eso no se cumple sin una regla para el codo.

Para una rótula (3 grados de libertad), el paper separa la dirección del eslabón (2 grados de libertad) del giro alrededor de su eje (1 grado de libertad). La dirección permitida es un cono irregular definido por cuatro ángulos. Si el punto queda afuera, se lo lleva al punto más cercano de la sección cónica (círculo, elipse o parábola), en el plano perpendicular al eje del cono. El giro se limita aparte, con la orientación de la articulación. Para articulaciones de un solo grado de libertad, como una bisagra, el paper propone un método más simple (Fig. 7):

1) La raíz y el objetivo, junto con la orientación del objetivo, definen un plano.
2) Las articulaciones se proyectan sobre ese plano.
3) Desde ahí el problema es 2D y FABRIK se aplica de la forma habitual, con forward y backward dentro del plano.

En los experimentos del paper con un modelo humano, esta técnica de bisagra es la que se usa para codos y rodillas (sección 5.3 y Fig. 19).

3.3. Resultados reportados en el paper

La Tabla 2 resume la Tabla 1 del paper, obtenida con una cadena de 10 articulaciones sin restricciones, un objetivo alcanzable y código MATLAB ejecutado en un Pentium 2 Duo de 2.2 GHz. Estos valores sirven para comparar los métodos entre sí, pero no para estimar tiempos en el ESP32.

Tabla 2. Resultados promedio (20 ejecuciones) de la Tabla 1 del paper [1].

Método	Iteraciones	Tiempo total	Tiempo por iteración
FABRIK	15.5	13.3 ms	0.86 ms
CCD	26.3	123.6 ms	4.69 ms
Jacobiano transpuesto	1311.2	12.99 s	9.90 ms
Jacobiano DLS	998.6	10.48 s	10.49 ms
Jacobiano SVD-DLS	808.8	9.30 s	11.50 ms

3.4. Limitación reportada

En la sección 5.4 del paper se describe un caso singular poco frecuente: la cadena está completamente recta y el objetivo cae sobre esa misma recta, entre dos articulaciones. El forward acerca la cadena a la raíz pero la deja recta, y el backward la devuelve a su lugar, de modo que el algoritmo entra en un ciclo. El paper propone doblar la cadena un ángulo muy pequeño, dentro de los límites, corriendo el objetivo durante la primera etapa backward y devolviéndolo después a su lugar. En nuestro brazo este caso puede aparecer cuando queda recta la cadena virtual que forman el hombro, el codo y $\boldsymbol{p}_5$ (Sección 5.3), y se trata en la Sección 7.

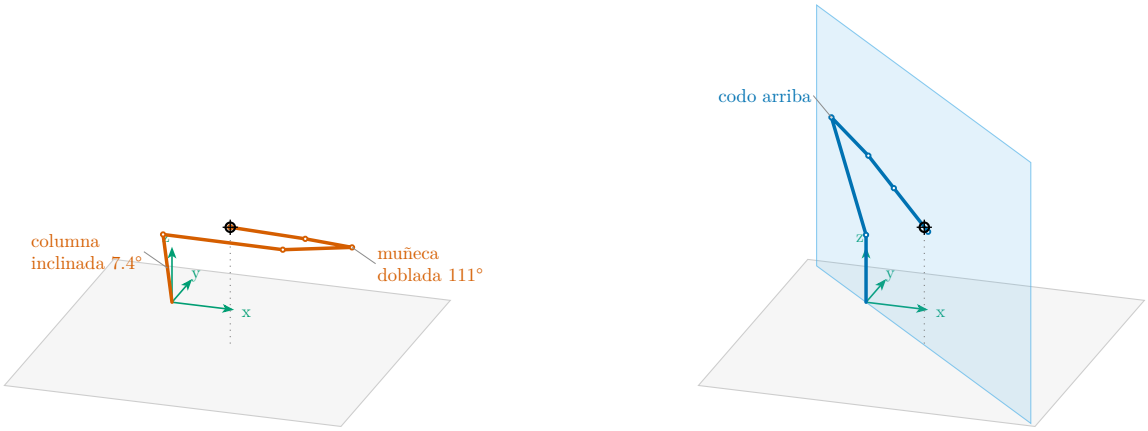
4. De rótulas libres a bisagras

4.1. El modelo general de la investigación

En la *Investigación de Modelos*, FABRIK se formaliza en su versión general: en 3D, sobre los seis puntos $\boldsymbol{p}_0, ..., \boldsymbol{p}_5$, como si cada articulación fuera una rótula libre. Es el comportamiento del Algoritmo 1 del paper sin restricciones, en el que la cadena se acomoda como un látigo. Allí mismo se aclara que, para nuestro brazo, las restricciones de bisagra deben aplicarse en cada paso y que los ángulos de los servos solo pueden recuperarse si las posiciones respetan esa estructura. En este documento, la aplicación directa del modelo general a nuestro brazo se denomina **v0**.

4.2. Aplicación directa a nuestro brazo

Con el objetivo $(5, -5, 7)$ y el brazo en la pose cero, FABRIK 3D libre converge en una sola iteración con un error prácticamente nulo. Sin embargo, el brazo no puede adoptar esa pose (Figura 3, Tabla 3).



(a) v0: FABRIK 3D con rótulas libres

(b) v3: FABRIK con bisagras, en el plano

Figura 3. El mismo objetivo resuelto (a) con rótulas libres y (b) con el método de bisagras. En (a) la cadena se acomoda como un látigo: se inclina la columna de la base y se doblan puntos donde no hay ninguna bisagra.

Tabla 3. Restricciones mecánicas que viola la solución con rótulas libres, objetivo $(5, -5, 7)$.

Qué se mide	En el modelo del brazo	FABRIK 3D libre
Columna de la base ($\boldsymbol{p}_0 \to \boldsymbol{p}_1$)	vertical: el servo solo gira	inclinada 7.4°
Hombro $\boldsymbol{p}_1$	fijo en $(0, 0, 4)$	se mueve a $(-0.51, -0.01, 3.97)$
Ángulo del antebrazo en $\boldsymbol{p}_3$	6.3°, fijo: la muñeca solo rota	111.4°

Como estas posiciones no respetan la estructura de bisagras, tampoco existen ángulos de servo que las reproduzcan. En la investigación también se advierte que medir el ángulo entre cada eslabón y su propio eje no sirve para recuperar los ángulos. La Tabla 4 lo comprueba con poses exactas, calculadas con la cinemática directa para que puntos y ejes sean coherentes:

Tabla 4. El ángulo entre cada eslabón y su propio eje, $\operatorname{atan2}(\|\boldsymbol{d}_i \times \boldsymbol{z}_i\|, \boldsymbol{d}_i \cdot \boldsymbol{z}_i)$, no depende de la pose.

Ángulos reales del modelo (°)	Resultado de la fórmula (°)
$(0, 0, 0, 0, 0)$	$(0, 90, 90, 180, 90)$
$(-30, 60, -90, 20, 0)$	$(0, 90, 90, 180, 90)$
$(-120, 100, -40, -60, 90)$	$(0, 90, 90, 180, 90)$

Esto pasa porque en una bisagra, como el hombro, el codo y la pinza, el eslabón siempre es perpendicular a su eje (90°), y en la base y la muñeca es paralelo (0° o 180°). Ninguno de esos valores cambia cuando gira el servo. El ángulo de una articulación debe medirse alrededor de su eje y con signo, como se hace en la Sección 5.5.

4.3. Por qué se eligió el método de bisagras

El método de bisagras coincide con la mecánica del brazo. Las figuras 2 y 4 del informe final del grupo J5 [5] muestran una base giratoria y dos bisagras paralelas. La muñeca rota sobre el eje del antebrazo y el servo 4 inclina la pinza. Con la muñeca y la pinza quietas, todo lo que está después del codo se comporta como un cuerpo rígido. Es un caso análogo al de la Fig. 7 del paper, que resuelve una bisagra pasando a un plano que contiene a la raíz y al objetivo. Proyectar sobre ese plano y resolver en 2D es la técnica que la sección 4.3 propone para bisagras.

Además, sigue siendo FABRIK iterativo. Dentro del plano se hacen las mismas etapas forward y backward, con los tres criterios de corte del paper, y lo único que se calcula de forma directa es la orientación del plano. En el paper, ese plano pasa por la raíz y por el objetivo. En nuestro brazo es siempre vertical y, cuando $\boldsymbol{p}_5$ sale del plano, queda a una distancia d del objetivo (Sección 5.3).

Para el ESP32 también es más ligero, porque cada iteración trabaja con tres puntos en 2D en lugar de seis puntos en 3D. Los ángulos salen directos y con signo, ya que en el plano cada uno es un atan2

entre dos direcciones conocidas. Por último, cada bisagra tiene un solo rango de ángulos, que se puede aplicar en cada paso como indica la sección 4.3.

5. FABRIK en el plano del brazo

El algoritmo completo se explica con el ejemplo que se usa en todo el informe, el objetivo $\boldsymbol{t} = (5, -5, 7)$ con la muñeca y la pinza neutras ($\theta_3 = 0°$, $\theta_4 = 90°$), que es la primera posición que prueba la búsqueda, y el hombro y el codo en 0°. A diferencia de la *Investigación de Modelos*, donde el forward y el backward son los pasos 1 y 2, aquí forman juntos el Paso 2.

5.1. Paso 0: alcanzabilidad

Antes de iterar se verifican dos condiciones:

1) Como $\boldsymbol{p}_5$ puede quedar a $|d|$ del plano del brazo (Sección 5.3), para que exista un plano que lo deje sobre el objetivo, la distancia horizontal del objetivo al eje de la base tiene que ser al menos $|d|$: $\rho = \sqrt{x^2 + y^2} \geq |d|$.
2) Dentro del plano del brazo, el objetivo tiene que quedar a lo sumo a $l_1 + l_2$ del hombro, $\|\boldsymbol{T}\| \leq l_1 + l_2$, con $\boldsymbol{T}$ definido en el Paso 1. Esa suma vale 18.91 cm con la muñeca y la pinza neutras. Con la muñeca neutra va de 15.49 cm, con la pinza en 0°, a 18.92 cm, con la pinza en 93.5°.

Si alguna de las dos falla, el objetivo está fuera de alcance con esas posiciones de la muñeca y la pinza, y se prueba la siguiente (Sección 9). Si no queda ninguna, no se itera y, como en el Algoritmo 1 del paper, se calcula la cadena estirada hacia el objetivo, sin tener en cuenta los límites de los servos. Esa cadena se muestra como silueta y el brazo no se mueve (Sección 8.3). En cambio, cumplir las dos condiciones no garantiza llegar respetando los límites, y eso se sabe recién durante la iteración.

5.2. Paso 1: el plano del brazo

Si $\boldsymbol{p}_5$ está dentro del plano, basta con apuntar la base al objetivo: $\varphi = \operatorname{atan2}(y, x)$. Si está a una distancia d hacia un costado, la base gira un poco distinto, para que el desvío lateral lo lleve exactamente al objetivo (Figura 4). Llamando r_c a la distancia dentro del plano:

$$\rho = \sqrt{x^2 + y^2}, \quad r_c = \pm\sqrt{\rho^2 - d^2}, \quad \theta_0 = \operatorname{atan2}(y, x) - \operatorname{atan2}(d, r_c)$$

Existen dos planos candidatos. Con $r_c > 0$ el brazo apunta hacia el objetivo, mientras que con $r_c < 0$ el plano queda girado al revés y el brazo tiene que inclinarse hacia atrás, por encima de la base. Se descartan los candidatos que dejan la base fuera de su rango, se prueba primero el plano que apunta al objetivo y el otro se usa solo si con el primero no se llega.

En el ejemplo, con la muñeca y la pinza neutras, $d = 0$: $r_c = \rho = 7.07$ cm y la base apunta directo al objetivo, $\theta_0 = \varphi = -45°$. El otro candidato, $\theta_0 = 135°$, queda fuera del rango de la base. Dentro del plano, el objetivo queda en $\boldsymbol{T} = (r_c, z - z_h) = (7.07, 3.00)$ cm respecto del hombro, donde $z_h = 4$ cm es la altura del hombro.

La Figura 4 muestra un caso con $d \neq 0$, el objetivo $(6, 1, 6)$. Apuntarle exigiría girar la base a $\varphi = 9.46°$, fuera de su rango, y con la muñeca neutra no se llega. La búsqueda, sin revisar choques, lo resuelve con la muñeca en 20° y la pinza en 140° (Sección 9): $\boldsymbol{p}_5$ queda $d = 1.13$ cm al costado del plano, $r_c = 5.98$ cm y la base gira a $\theta_0 = -1.21°$, dentro de su rango.

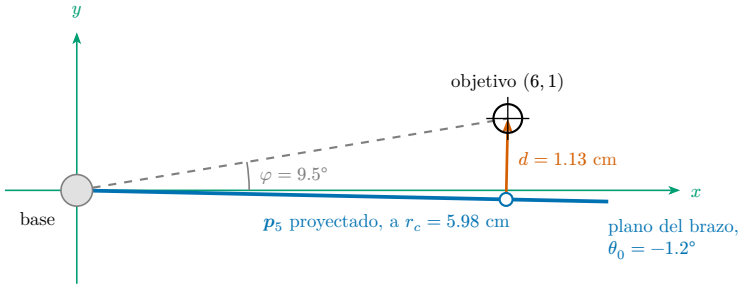


Figura 4. Vista superior del Paso 1 para el objetivo $(6, 1, 6)$, con la muñeca en 20° y la pinza en 140°. La línea gris rayada apunta directo al objetivo, a 9.5°, fuera del rango de la base (−180° a 0°). Como $\boldsymbol{p}_5$ queda 1.13 cm al costado del plano, la base gira a −1.2° y el desvío lateral lo lleva al objetivo.

5.3. La cadena virtual hombro, codo, p_5

Dentro del plano, FABRIK trabaja con tres puntos: el hombro (raíz fija), el codo y $\boldsymbol{p}_5$. Para posiciones dadas de la muñeca y la pinza, todo lo que está después del codo (antebrazo, muñeca y pinza) se comporta como un cuerpo rígido y se puede reemplazar por un único segmento virtual desde el codo hasta $\boldsymbol{p}_5$. Ese segmento depende de la muñeca θ_3 y de la pinza θ_4:

$$l_1 = 6.9 \text{ cm}, \quad \boldsymbol{v} = (7.7 + 4.3\sin\theta_4, 0.5 - 4.3\cos\theta_3\cos\theta_4), \quad l_2 = \|\boldsymbol{v}\|,$$
$$\alpha = \operatorname{atan2}(v_z, v_r), \quad d = -4.3\sin\theta_3\cos\theta_4$$

donde $\boldsymbol{v}$ es la parte contenida en el plano del segmento que va del codo a $\boldsymbol{p}_5$, α es el ángulo fijo entre ese segmento y el antebrazo, y d es la distancia de $\boldsymbol{p}_5$ al plano. Con la muñeca neutra, $d = 0$ con cualquier pinza: la pinza mueve $\boldsymbol{p}_5$ dentro del plano. Con la pinza alineada, $\boldsymbol{v} = (12.0, 0.5)$ y $d = 0$ con cualquier muñeca, porque $\boldsymbol{p}_5$ queda sobre el eje de la muñeca.

Tabla 5. Parámetros de la cadena virtual según la muñeca y la pinza. Con la muñeca neutra la pinza mueve $\boldsymbol{p}_5$ dentro del plano ($d = 0$); con la muñeca girada y la pinza inclinada, lo saca del plano.

Muñeca θ_3	Pinza θ_4	l_2 (cm)	α	d (cm)	p_5 con los demás ángulos en 0
0°	0°	8.59	−26.27°	0	$(14.6, 0, 0.2)$: 4.3 cm abajo de $\boldsymbol{p}_4$
0°	90°	12.01	2.39°	0	$(18.9, 0, 4.5)$: 4.3 cm adelante, alineada con el antebrazo
0°	180°	9.07	31.94°	0	$(14.6, 0, 8.8)$: 4.3 cm arriba
90°	0°	7.72	3.72°	−4.3	$(14.6, -4.3, 4.5)$: 4.3 cm a la derecha

5.4. Paso 2: iteración dentro del plano

Dentro del plano se aplican las dos etapas del paper sobre los tres puntos: hombro $\boldsymbol{S}$ (fijo en el origen del plano), codo $\boldsymbol{E}$ y punta $\boldsymbol{P}$ ($\boldsymbol{p}_5$ proyectado):

- **Forward:** $\boldsymbol{P} \leftarrow \boldsymbol{T}$. El codo se mueve sobre la recta que lo une con $\boldsymbol{P}$ hasta quedar a l_2: $\boldsymbol{E} \leftarrow \boldsymbol{P} + l_2(\boldsymbol{E} - \boldsymbol{P})/\|\boldsymbol{E} - \boldsymbol{P}\|$.
- **Backward:** el hombro vuelve a su lugar ($\boldsymbol{S} = \boldsymbol{0}$). El codo se mueve sobre la recta que lo une con el hombro hasta quedar a l_1, y la punta, sobre la recta que la une con el codo, hasta quedar a l_2.

En el paper, el forward también mueve la raíz. Con tres puntos ese paso no cambia el resultado, porque el backward devuelve el hombro a su lugar y solo usa la posición del codo que dejó el forward. En cada paso se aplican las restricciones de la Sección 8: el lado del codo en el forward y los rangos del hombro y del codo en el backward.

5.5. Paso 3: recuperación de los ángulos

Con los puntos finales del plano (r horizontal, h vertical, medidos desde el hombro):

$$\theta_1 = \operatorname{atan2}(E_h, E_r), \quad \theta_2 = \operatorname{atan2}(P_h - E_h, P_r - E_r) - \alpha - \theta_1$$

donde θ_1 es la inclinación del brazo respecto de la horizontal y θ_2 es el giro del antebrazo respecto del brazo; se resta α porque el segmento virtual no está alineado con el antebrazo. Ambos ángulos tienen signo. θ_0 se obtiene en el Paso 1, y θ_3 y θ_4 son las posiciones de la muñeca y la pinza con las que se está probando.

5.6. Ejemplo completo

La Tabla 6 muestra cómo evoluciona la cadena para el objetivo de ejemplo.

Tabla 6. Iteraciones para el objetivo $(5, -5, 7)$ con la versión v3. Coordenadas en cm dentro del plano, medidas desde el hombro.

k	Forward	Backward	Error
0	—	hombro y codo en 0°: codo $(6.90, 0)$, punta $(18.90, 0.50)$	12.09 cm
1	codo en $(6.39, -8.99)$: lado prohibido, se espeja a $(-2.03, 10.84)$	codo $(-1.27, 6.78)$, punta $(9.67, 1.82)$	2.85 cm
2	codo en $(-3.87, 7.96)$	codo $(-3.01, 6.21)$, punta $(8.43, 2.57)$	1.43 cm
3	codo en $(-4.37, 6.64)$	codo $(-3.80, 5.76)$, punta $(7.84, 2.80)$	0.80 cm
4	codo en $(-4.57, 5.96)$	codo $(-4.20, 5.48)$, punta $(7.53, 2.90)$	0.47 cm

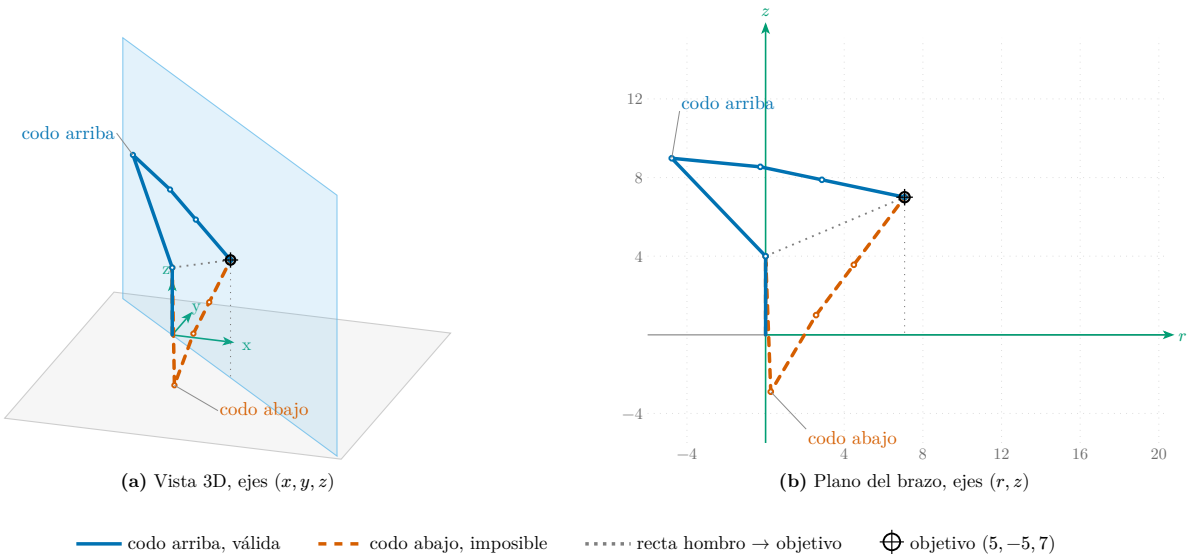
Después del espejo de la primera iteración el codo ya queda del lado permitido, y en las siguientes se acomoda hasta que el error baja de $\varepsilon = 0.5$ cm en la cuarta. Los ángulos resultantes son $(\theta_0, \theta_1, \theta_2) = (-45°, 127.49°, -142.26°)$, y al aplicarles la cinemática directa se obtiene $\boldsymbol{p}_5 = (5.33, -5.33, 6.90)$, a 0.47 cm del objetivo. La cadena virtual mide 12.01 cm, casi el doble que el brazo, así que para un objetivo tan cercano al hombro el codo se pliega mucho: el brazo queda inclinado hacia atrás y el antebrazo vuelve hacia el objetivo.

6. El problema del codo abajo

6.1. Dos maneras de llegar al mismo punto

El codo del modelo va de −180° a 0°, así que el antebrazo solo puede caer respecto del brazo. Sin embargo, para llegar a un mismo punto, el triángulo que forman el hombro, el codo y la punta se puede armar de dos maneras: con el codo por arriba o por debajo de la recta que une el hombro con el objetivo. Ambas tienen las mismas longitudes y ubican $\boldsymbol{p}_5$ exactamente sobre el objetivo, pero el brazo solo puede hacer una:

- **Codo arriba** ($\theta_1 = 133.79°$, $\theta_2 = -145.67°$): válida.
- **Codo abajo** ($\theta_1 = -87.81°$, $\theta_2 = +140.90°$): imposible, porque el codo doblaría al revés y el hombro bajaría de 0°.



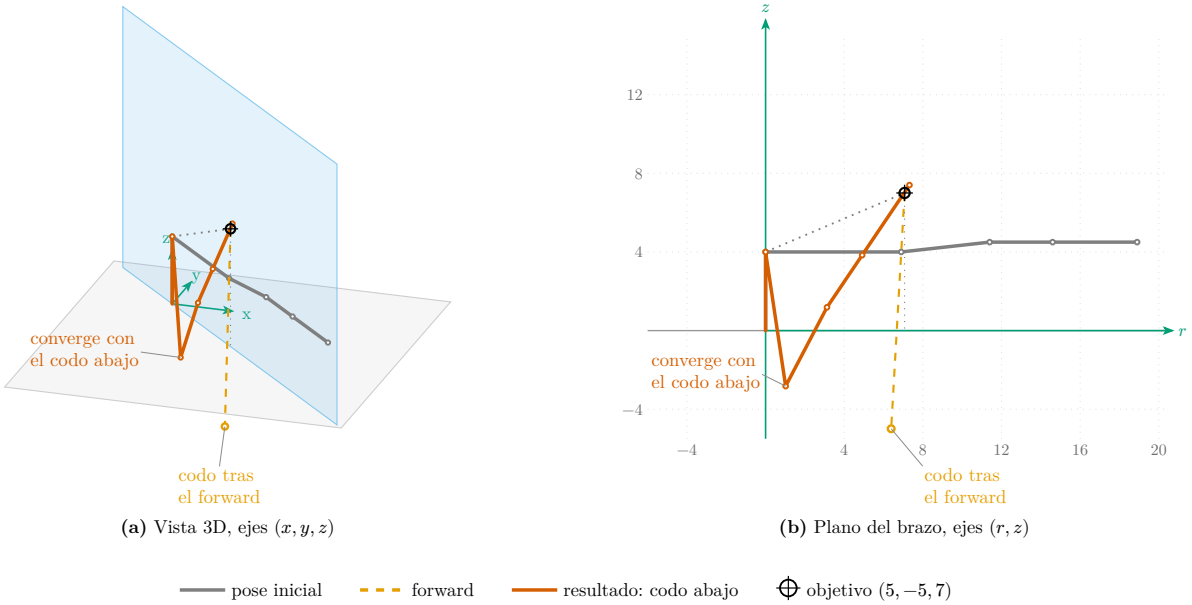
(a) Vista 3D, ejes (x, y, z) **(b)** Plano del brazo, ejes (r, z)

Figura 5. Las dos soluciones geométricas para el objetivo $(5, -5, 7)$. Con la pinza alineada, la mano sigue la dirección del antebrazo hasta el objetivo.

6.2. Qué hace FABRIK sin una regla para el codo

FABRIK no distingue cuál de las dos soluciones es la permitida. En general converge a la del lado en que el primer forward deja el codo, que depende de la pose inicial. Partiendo del hombro y el codo en 0°, con el brazo estirado y horizontal:

1) El primer forward coloca $\boldsymbol{p}_5$ sobre el objetivo y arrastra el codo sin cambiarlo de lado. El codo queda en $(6.39, -8.99)$, por debajo de la recta hombro→objetivo.
2) El backward conserva ese lado y el algoritmo converge en cuatro iteraciones con un error de 0.47 cm, pero con el codo abajo: $\theta_1 = -81.51°$ y $\theta_2 = +137.49°$, ambos fuera de rango (Figura 6).



(a) Vista 3D, ejes (x, y, z) **(b)** Plano del brazo, ejes (r, z)

Figura 6. FABRIK sin regla para el codo, partiendo del hombro y el codo en 0°, con la base ya girada en el Paso 1. El primer forward deja el codo del lado prohibido y la solución converge ahí.

6.3. Ajustar los ángulos al final no resuelve el problema

Una alternativa sería ajustar cada ángulo a su rango después de converger y declarar inválida la pose si hubo que ajustar alguno. En el ejemplo, el hombro pasa de −81.51° a 0°, el tope más cercano medido en ángulo, y el codo de 137.49° a −180°. El brazo queda plegado sobre sí mismo, con $\boldsymbol{p}_5$ a 12.66 cm del objetivo. La pose se declara inválida, aunque el objetivo era alcanzable con el codo arriba.

6.4. Limitar en cada paso tampoco alcanza

Si se aplican los rangos en cada paso pero no se controla el lado del codo, el resultado depende de hacia qué tope cae el recorte. En el ejemplo, el primer backward deja el hombro en −54.61°, más cerca del tope de 0° que del de 180°, y el recorte lo lleva a 0°. El brazo queda recto, el error se mantiene en 12.09 cm y, como esa iteración deja $\boldsymbol{p}_5$ donde estaba, el algoritmo corta por falta de mejora. Con el objetivo un centímetro más arriba, en $(5, -5, 8)$, pasa lo mismo: el primer backward deja el hombro en −50.66°, el recorte lo lleva a 0° y el error queda en 12.34 cm. La cadena queda trabada en la pose recta porque choca con el límite del codo por el lado equivocado.

6.5. Magnitud del problema

Para medirlo se generaron 2000 objetivos alcanzables a partir de ángulos válidos elegidos al azar, aplicando la cinemática directa, de modo que se sabe que cada uno tiene solución. Cada variante se probó con la muñeca y la pinza con las que se generó cada objetivo, partiendo de la pose cero y de una pose al azar (Tabla 7). En esta medición, la pose cero tiene el hombro y el codo en 0°, y la pose al azar, esos dos ángulos sorteados dentro de su rango. Al aplicar los límites al final se pierde más de la mitad de los objetivos alcanzables desde la pose cero. Aplicarlos en cada paso mejora poco si no se usa la regla del codo, y con la regla convergen casi todos.

Tabla 7. Objetivos resueltos (error ≤ 0.5 cm y ángulos en rango) sobre 2000 objetivos alcanzables. Simulación en Python, con los límites del firmware (Tabla 9).

Variante	Desde la pose cero	Desde pose al azar
v1: plano sin límites y chequeo al final	723 (36 %)	1383 (69 %)
v1 + espejo del codo una sola vez, al final	1947 (97 %)	1902 (95 %)
v2: límites en cada paso, sin regla del codo	823 (41 %)	1451 (73 %)
v3: límites en cada paso y regla del codo	1980 (99 %)	1980 (99 %)

7. La solución: la técnica *pole* de Blender y Maya

7.1. El problema en animación

Este problema también aparece en animación 3D. Cuando se mueve la mano de un personaje con cinemática inversa, la mano define dónde termina el brazo, pero no hacia dónde apunta el codo. Con el hombro y la mano fijos, el codo puede estar en cualquier punto de una circunferencia alrededor de la recta que une el hombro con la mano. Para elegir uno, los programas de animación usan un *pole*, que es un punto o una dirección auxiliar que indica hacia dónde debe apuntar el codo.

- **Blender.** La restricción de cinemática inversa tiene la opción *Pole Target*. El manual la define como el objeto que determina el giro (*roll*) de la cadena; en un brazo, determina dónde queda el codo [2].
- **Maya.** El solver *Rotate Plane* mantiene la cadena (hombro, codo y muñeca) en un plano que gira alrededor del vector que va del inicio al final de la cadena. El *pole vector* controla la orientación de ese plano [3]. La restricción `poleVectorConstraint` hace que el pole vector apunte a un objeto, de modo que ese objeto quede en el plano de rotación [4]. La documentación advierte que la cadena puede invertirse de golpe (*flipping*) si el vector de la cadena y el pole vector se cruzan o quedan opuestos [3].

7.2. Cómo se traduce a nuestro brazo

Como en nuestro brazo la base ya fijó el plano (Paso 1), de toda la circunferencia de posibles codos solo quedan los dos puntos donde la circunferencia corta al plano. Esos dos puntos son simétricos respecto de la recta hombro→objetivo y corresponden a las dos soluciones de la Figura 5. El pole lleva el codo al punto opuesto de la circunferencia, girándolo 180° alrededor de esa recta. Dentro del plano, ese giro equivale a *espejar* el codo respecto de la recta.

El pole de nuestro brazo es fijo, porque el codo siempre tiene que quedar del mismo lado de la recta hombro→objetivo, que es el de arriba cuando el objetivo está delante del hombro. En cada forward se aplica en dos pasos:

1) Se calcula de qué lado de la recta quedó el codo, con el producto vectorial en 2D:

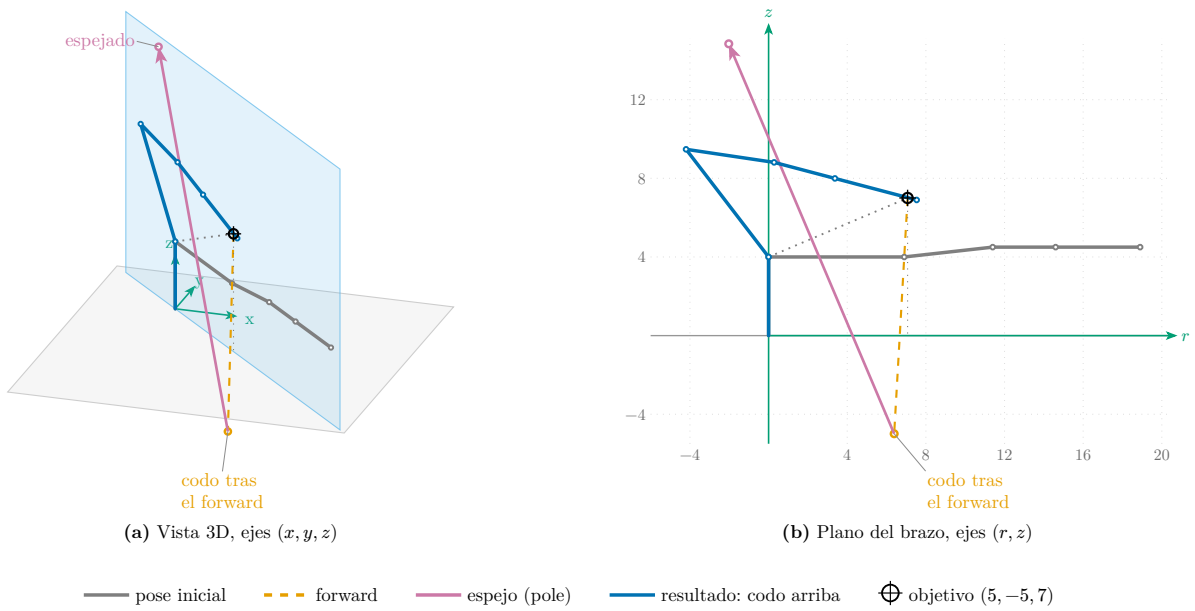
$$s = T_r E_h - T_h E_r$$

   Si $s > 0$ el codo está del lado permitido, y si $s < 0$, del lado prohibido.

2) Si está del lado prohibido, se lo espeja respecto de la recta:

$$\boldsymbol{E}' = 2(\boldsymbol{E} \cdot \hat{\boldsymbol{u}})\,\hat{\boldsymbol{u}} - \boldsymbol{E}, \quad \hat{\boldsymbol{u}} = \frac{\boldsymbol{T}}{\|\boldsymbol{T}\|}$$

El espejo no altera el funcionamiento de FABRIK. El hombro (origen) y el objetivo están sobre la recta, y una reflexión respecto de una recta conserva las distancias a los puntos de esa recta. Por lo tanto, $\|\boldsymbol{E}'\| = \|\boldsymbol{E}\|$ y $\|\boldsymbol{E}' - \boldsymbol{T}\| = \|\boldsymbol{E} - \boldsymbol{T}\| = l_2$. El forward sigue siendo válido y solo cambia el lado del codo. La Figura 7 muestra el resultado en el ejemplo.



(a) Vista 3D, ejes (x, y, z) **(b)** Plano del brazo, ejes (r, z)

Figura 7. FABRIK con la regla del codo. El codo que dejó el forward se espeja respecto de la recta hombro→objetivo (punteada) y, tres iteraciones después, el brazo llega a 0.47 cm del objetivo con el codo arriba.

7.3. Detalles de la regla

En Maya el pole vector suele apuntar a un objeto de la escena [4], y la cadena puede invertirse cuando el vector de la cadena lo cruza o queda opuesto a él [3]. En nuestro brazo el pole es "el lado permitido de la recta hombro→objetivo" y se define respecto de esa misma recta, así que nunca queda alineado con ella y no se producen vuelcos.

El ángulo α corre las fronteras de la regla. Con la cadena virtual, $s = l_1 l_2 \sin(-(\theta_2 + \alpha))$. Entonces s se anula en $\theta_2 = -\alpha$ y en $\theta_2 = -180° - \alpha$, y no en $\theta_2 = 0$. Si el codo está casi recto, con θ_2 entre $-\alpha$ y $0°$, la configuración es válida pero queda apenas del lado prohibido. La regla la reemplaza por su espejo, $\theta_2' = -\theta_2 - 2\alpha$, que también es válida y llega al mismo punto con el codo un poco más doblado. Algo parecido ocurre con el codo casi plegado cuando, con la muñeca neutra, la pinza baja de unos $83.3°$, porque entonces $\alpha < 0$ y la segunda frontera queda dentro del rango del codo. En ese caso el espejo, tomado módulo $360°$, vale $\theta_2' = -\theta_2 - 2\alpha - 360°$. Los dos casos pueden importar cerca de los topes.

Por último está la cadena recta de la sección 5.4 del paper. Si el codo cae sobre la recta, s vale 0 y espejarlo no lo mueve. Entonces el efector no avanza, la iteración corta por η y la búsqueda de la Sección 9 pasa a otra posición. Como cada posición de la pinza tiene su propio α, con la siguiente la cadena virtual ya no queda recta y FABRIK sigue normalmente. Por esta razón, no hace falta la corrección que propone el paper.

8. Restricciones articulares y validación

8.1. Restricciones en cada paso

Siguiendo la sección 4.3 del paper, las restricciones se aplican dentro de cada iteración y no al final. A ese criterio se suma la regla del codo, que no forma parte del paper:

- **Forward:** después de ubicar el codo, se aplica la regla del codo (Sección 7).
- **Backward:** después de ubicar el codo a l_1 del hombro, si la inclinación del brazo sale de su rango (hombro de 0° a 180°), se usa el límite más cercano y se recalcula el codo. Después de ubicar la punta a l_2 del codo, si el giro relativo $\theta_2 + \alpha$ sale de $[-180° + \alpha, \alpha]$, se usa el límite más cercano y se recalcula la punta.

El límite más cercano se mide en ángulo. Por ejemplo, un hombro en $-170°$ está a 10° del tope de 180° y a 170° del de 0°, así que se lleva a 180°.

Los límites son los que permite el firmware, se expresan en grados del servo y se convierten al modelo (Sección 10).

8.2. Criterios de corte

Se usan los tres criterios del paper, en este orden:

Tabla 8. Parámetros de corte de la iteración.

Parámetro	Valor	Significado
ε (tolerancia)	0.5 cm	Corta cuando $\|\boldsymbol{p}_5 - \boldsymbol{t}\| \leq \varepsilon$. Con las medidas del brazo, un grado en el hombro mueve la punta hasta 0.33 cm, y se supone que 0.5 cm cubre además el juego de los engranajes de los SG90.
η (sin cambio)	10^{-4} cm	Corta cuando $\boldsymbol{p}_5$ se mueve menos que η entre dos iteraciones. Es el caso de un objetivo que no se alcanza por los límites; la cadena recta de la sección 5.4 del paper también lo activa, y entonces la búsqueda prueba la siguiente posición (Sección 7).
$k_{\max}$ (iteraciones)	20	Tope de seguridad. En la medición de la Tabla 7 el promedio es 3.1 iteraciones. Sin embargo, en la búsqueda de la Sección 9 una de cada doce ejecuciones que no llegan termina por este criterio, y casi la mitad de ellas porque la cadena sigue deslizándose contra un tope del codo.

8.3. Validación final, silueta y colisiones

Al terminar la iteración se hace el control final que plantea la investigación: todos los ángulos deben estar en rango y el error debe ser $\leq \varepsilon$. Con las restricciones aplicadas en cada paso este control normalmente se cumple, pero se mantiene como protección. Si no se cumple, se prueba el otro plano candidato y, si tampoco llega, la siguiente posición de la pinza o de la muñeca (Sección 9).

Si la pose es válida, se evalúa el modelo de esferas heredado del grupo J5, que detecta autocolisiones entre eslabones y contacto con el piso. Si hay colisión, la pose se descarta igual que una que no llega: se prueba el otro plano y, después, la siguiente posición de la pinza o de la muñeca. Solo una pose válida y sin colisiones pasa al movimiento suavizado.

Si ninguna posición llega sin chocar, el brazo no se mueve, y la simulación dibuja en gris el mejor intento con uno de tres mensajes. El mensaje «colisión» aparece cuando alguna posición llega, pero todas las que llegan chocan, y se dibuja la primera de ellas con las esferas que chocan en rojo. El mensaje «fuera de alcance» aparece cuando el Paso 0 falló con todas las posiciones probadas. En ese caso se dibuja la cadena estirada hacia el objetivo con la muñeca y la pinza neutras, como en el Algoritmo 1 del paper, y puede no respetar los límites de los servos.

El mensaje «bloqueado por límites» aparece cuando con alguna posición la distancia alcanza, pero los límites impiden llegar. En ese caso se dibuja el intento más cercano, que respeta todos los límites. Para elegir ese intento se toma la pose de menor error de todas las iteraciones, porque una ejecución que

termina por $k_{\max}$ puede quedar más lejos del objetivo que en una iteración anterior. La Figura 8 resume el recorrido completo.

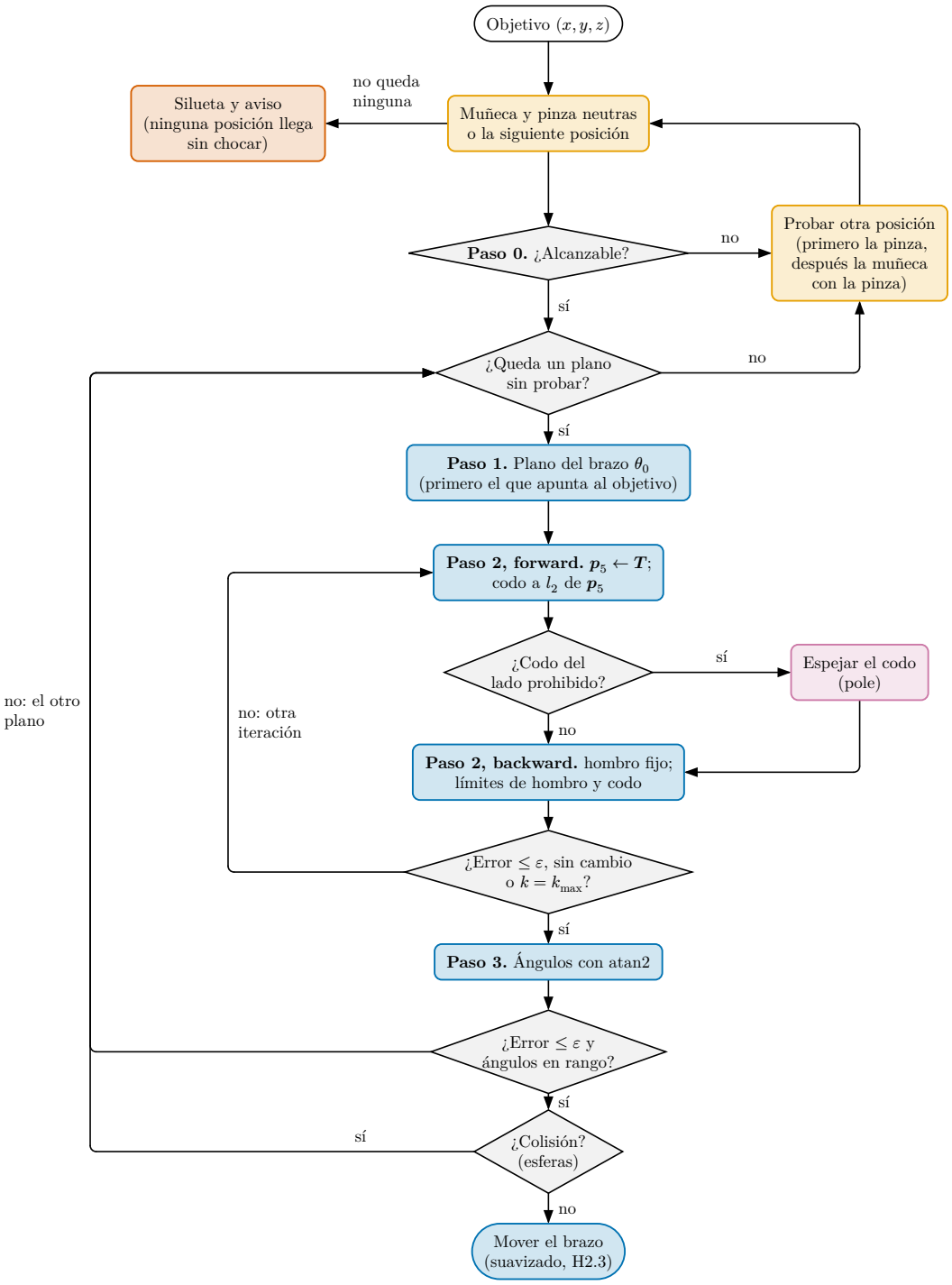


Figura 8. Flujo completo, desde que el usuario carga el objetivo hasta que el brazo se mueve.

9. Qué calcula FABRIK y cómo se eligen la muñeca y la pinza

El usuario solo indica el punto (x, y, z) al que tiene que llegar la pinza. Con tres coordenadas se pueden determinar tres ángulos. Por lo tanto, FABRIK calcula la base (θ_0, en el Paso 1), el hombro (θ_1) y el codo (θ_2). Los otros dos servos los elige el programa: la muñeca (servo 3), que gira la pinza alrededor del eje del antebrazo, y el servo 4, que es la pinza y se inclina de 0° a 180°.

FABRIK necesita esos dos valores antes de iterar, porque de ellos dependen l_2, α y d (Tabla 5) y, a través de d, también el ángulo de la base. Si tuviera que calcularlos, habría cinco incógnitas para tres ecuaciones y el resultado dependería de la pose de partida. Por esta razón, el programa los elige con una búsqueda:

1) Prueba la muñeca en $\theta_3 = 0°$ (servo en 90°) y la pinza alineada con el antebrazo.
2) Si el objetivo queda fuera de alcance o bloqueado por los límites, prueba otras posiciones de la pinza de a 10°, de la más cercana a la neutra a la más lejana (+10°, −10°, +20°, −20°, y así hasta los topes).
3) Si ninguna llega, prueba cada posición de la muñeca, en el mismo orden, con todas las de la pinza.
4) Se queda con la primera posición que llega. Si ninguna llega, el brazo no se mueve (Sección 8.3).

La pinza va primero porque inclinarla corre su punta dentro del plano del brazo. La muñeca va después y siempre junto con la pinza: con la pinza alineada, $\boldsymbol{p}_5$ queda sobre el eje de la muñeca y girarla no lo mueve. Con la pinza inclinada, girar la muñeca saca la punta del plano, y eso permite llegar a puntos a los que la base no puede apuntar, como el de la Figura 4. Una posición con la misma cadena virtual que otra que ya falló se saltea sin iterar, como pasa con la muñeca girada y la pinza alineada.

Para medir la búsqueda se usaron los mismos 2000 objetivos de la Tabla 7, partiendo del home del firmware, sin revisar choques y sin usar la muñeca ni la pinza con las que se generó cada uno. Con la muñeca y la pinza neutras se resuelve el 84.95 %, y con la búsqueda, el 99.75 %: 213 objetivos más con la pinza inclinada y 83 con la muñeca girada y la pinza. La mediana es de 2 iteraciones por objetivo, y los pocos objetivos que necesitan muchas posiciones suben el promedio a 6.7 posiciones y 24.1 iteraciones.

Por ejemplo, $(3, -3, 0)$ está sobre el piso, cerca de la base. Con la pinza alineada, la cadena virtual es demasiado larga para un punto tan cercano al hombro y, aun con el codo plegado del todo, la punta queda a 0.68 cm. La búsqueda lo resuelve con la pinza en 30°, a 0.41 cm, después de 13 posiciones. Quedan cinco objetivos sin resolver. Uno, $(0.71, 0.16, 22.90)$, está casi sobre el eje de la base y en el borde del alcance. Se generó con la pinza en 93.2°, cerca de los 93.5° del alcance máximo (Sección 5), y la búsqueda, que prueba la pinza de a 10°, no pasa por esos ángulos. Los otros cuatro están del lado $+y$, a 3 o 4 cm del eje de la base. La base no puede apuntarles y, con el brazo inclinado hacia atrás, los límites no lo dejan llegar.

10. Convención de ángulos y límites

Se trabaja con dos sistemas de grados:

- **Grados del servo** (s, de 0 a 180°): el ángulo de cada SG90 que el firmware obtiene del ancho de pulso con las tablas de `main.ino`.
- **Grados del modelo** (θ): los que usan la cinemática directa, las colisiones y FABRIK.

El firmware convierte de uno a otro invirtiendo el signo de la base y del codo y restando 90° a la muñeca. La implementación usa la misma conversión y expresa los límites en grados del servo (Tabla 9).

Tabla 9. Límites de cada servo según `main.ino` y conversión a ángulos del modelo.

Servo	Pulso en `main.ino`	Rango del servo	Conversión	Rango del modelo
Base (0)	400 a 2600 µs	0 a 180°	$\theta_0 = -s_0$	−180° a 0°
Hombro (1)	400 a 1800 µs	0 a 180°	$\theta_1 = s_1$	0° a 180°
Codo (2)	400 a 2600 µs	0 a 180°	$\theta_2 = -s_2$	−180° a 0°
Muñeca (3)	400 a 2000 µs	0 a 180°	$\theta_3 = s_3 - 90°$	−90° a 90°
Pinza (4)	400 a 2600 µs	0 a 180°	$\theta_4 = s_4$	0° a 180°

Los rangos son los que permite el firmware del grupo J5. `main.ino` limita el ancho de pulso de cada servo y, con sus tablas, ese ancho equivale a un rango de 0° a 180° en los cinco.

11. Versionado del modelo

El modelo pasó por las versiones de la Tabla 10 y la Figura 9. Los porcentajes corresponden a los objetivos resueltos sobre 2000 objetivos alcanzables, partiendo de la pose cero y de una pose al azar (Tabla 7).

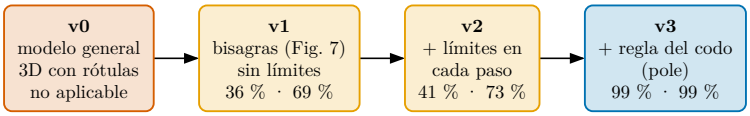


Figura 9. Versiones del modelo de FABRIK. Los porcentajes son: desde la pose cero · desde pose al azar.

Tabla 10. Versiones del modelo de FABRIK.

Versión	Qué hace	Problema o resultado	Estado
v0	Formulación general de la investigación, sin las restricciones de bisagra: FABRIK 3D sobre $\boldsymbol{p}_0, ..., \boldsymbol{p}_5$ como rótulas libres	Poses imposibles (base inclinada, muñeca doblada) que ningún conjunto de ángulos de servo reproduce	Descartada
v1	Método de bisagras: Paso 1 (plano) y FABRIK 2D, sin restricciones; chequeo de límites al final	Converge al codo abajo según la pose inicial: 36 % y 69 %	Descartada
v2	v1 con restricciones de hombro y codo en cada paso (sec. 4.3)	Se traba recto al chocar el límite del codo por el lado equivocado: 41 % y 73 %	Descartada
v3	v2 con la regla del codo (pole) en cada forward	99 % y 99 %; 3.1 iteraciones en promedio; unos 0.1 ms por objetivo en Python, según la máquina	Adoptada

Para la medición, cada objetivo se generó eligiendo al azar los cinco ángulos dentro del rango de la Tabla 9 y calculando $\boldsymbol{p}_5$ con la cinemática directa. Un objetivo se considera resuelto si el error es ≤ 0.5 cm, los ángulos están en rango y la cinemática directa de esos ángulos deja $\boldsymbol{p}_5$ a 0.5 cm o menos del objetivo. La medición usa una semilla aleatoria fija, y `test_medicion.py` la repite con la implementación definitiva.

12. Plan de implementación y pruebas

12.1. Simulación en Python

- **`fabrik.py`** (nuevo): la función principal recibe el objetivo, los ángulos actuales y los límites, y devuelve los ángulos (incluidas la muñeca y la pinza elegidas), el error, las iteraciones y un estado (`ok`, `fuera_de_alcance` o `bloqueado_por_limites`). Internamente separa la búsqueda de la muñeca y la pinza, el Paso 1 (planos candidatos), la iteración en el plano con la regla del codo y las restricciones, y la recuperación de ángulos.
- **`configuracion.py`**: los límites del firmware en grados del servo y las funciones de conversión servo ↔ modelo (Tabla 9).
- **`test_fabrik.py`** (nuevo):
  - FK(IK($\boldsymbol{p}$)) $\approx \boldsymbol{p}$ sobre objetivos generados con ángulos válidos;
  - objetivos fuera de alcance;
  - ángulos siempre dentro de rango;
  - el ejemplo $(5, -5, 7)$ como caso fijo;
  - la búsqueda de la muñeca y la pinza, con $(3, -3, 0)$ y $(6, 1, 6)$ como casos fijos (se resuelven con la pinza en 30° y con la muñeca en 20° y la pinza en 140°);
  - cadena recta;
  - cantidad de iteraciones y tiempo.
- **Interfaz:** ventana de matplotlib con:
  - el cuadro de texto x, y, z;
  - doble clic sobre un *plano de trabajo* horizontal, cuya altura se elige en la interfaz. El clic define una recta de visión, y su intersección con ese plano da el punto;
  - la silueta gris con mensajes.

  La consola se usa solo para mensajes, ya que `input()` congela la ventana.

12.2. Pasaje a C (ESP32)

Una vez validada la simulación, se porta respetando las reglas de firmware de la cátedra:

- sin memoria dinámica (`malloc`/`free`), con arreglos de tamaño fijo;
- sin `delay()`;
- máquina de estados dentro del super-loop disparado por tiempo;
- cada tarea dentro del tiempo de tick (WCET); si la búsqueda no entra en un tick, se reparte entre ciclos de la máquina de estados.

Como la iteración trabaja con tres puntos en 2D, el costo por iteración es de unas pocas operaciones de punto flotante y una raíz cuadrada por punto.

12.3. Pendientes

- Medir el desvío de 0.5 cm entre el eje del codo y el del antebrazo, la única medida que todavía viene del grupo J5, y pasar todas las medidas a `configuracion.c` con el pasaje a C.
- Rehacer las esferas de colisión para el brazo medido: las del grupo J5 se pensaron para las medidas de `configuracion.c` y dejan partes del brazo sin cubrir.
- Medir qué ángulo del servo 4 deja la pinza alineada con el antebrazo, que es su posición neutra.
- Confirmar el sentido de giro de la base y de la muñeca, y medir los topes reales de los cinco servos (FABRIK usa los del firmware).

12.4. Mejora a futuro: la pinza dentro de FABRIK

Esta mejora se evalúa recién cuando FABRIK con tres puntos esté validado en la simulación.

Con la muñeca en su posición neutra, el eje de la pinza queda paralelo a los del hombro y el codo, así que la pinza es una tercera bisagra dentro del plano del brazo. La cadena del plano podría pasar de tres puntos a cuatro (hombro, codo, eje de la pinza y su punta), y FABRIK calcularía la base, el hombro, el codo y la pinza, cuatro de los cinco servos. El paper hace sus pruebas con cadenas de ese tipo, con más articulaciones que coordenadas del objetivo, como la de 10 articulaciones de su Tabla 1 (Tabla 2). En animación es el uso habitual. Unreal Engine trae un nodo FABRIK [6] y Final IK lo incluye para Unity

[7]. En robots industriales, en cambio, la cinemática inversa suele ser analítica o numérica, y el objetivo incluye la orientación de la herramienta [8].

La muñeca queda afuera. Gira alrededor del eje del antebrazo y por eso no es una bisagra del plano. Al girar inclina el eje de la pinza respecto de los del hombro y el codo, y la cadena deja de ser plana. Para incluirla habría que pasar a FABRIK en 3D con la orientación de cada articulación (Algoritmo 2 del paper), y se perderían el plano, la regla del codo y el bajo costo para el ESP32, sin ganar nada cuando el objetivo es solo un punto.

Con cuatro puntos aparecen problemas que con tres no existen:

1) Dentro del plano hay tres bisagras para dos coordenadas, y por eso el mismo punto tiene infinitas soluciones y FABRIK elige según la pose de partida. Un mismo objetivo podría terminar con la pinza doblada de distinta forma según de dónde venga el brazo. Se puede evitar arrancando siempre desde una pose de referencia con la pinza alineada con el antebrazo, o agregando una preferencia por la pinza recta.
2) La regla del codo se pensó para el triángulo que forman el hombro, el codo y la punta. Con cuatro puntos el codo puede quedar del lado prohibido de otras maneras, y la pinza puede plegarse hacia atrás dentro de su rango de 0° a 180°.
3) Cada bisagra con límites es otro lugar donde la cadena puede quedar trabada, como pasaba en la v2. El 99 % corresponde a la cadena de tres puntos y habría que medirlo de nuevo.
4) FABRIK puede llegar al punto con el codo casi recto y la pinza muy doblada, una pose válida pero poco natural. La búsqueda de la Sección 9, en cambio, deja la pinza recta siempre que puede.

El punto extra no cambia el costo de forma apreciable. Para decidir, las dos opciones se van a comparar con 2000 objetivos alcanzables, midiendo el porcentaje resuelto, las iteraciones, si el mismo punto da siempre la misma pose y si las poses se ven naturales. Si la cadena de cuatro puntos resulta mejor, pasa a ser el método (v4) y la búsqueda queda solo para la muñeca, como último recurso.

13. Conclusiones

1) Nuestro brazo no se comporta como una cadena de rótulas libres, ya que tiene una base giratoria y dos bisagras paralelas. Por eso, aplicar FABRIK 3D libre produce poses que los servos no pueden hacer.
2) El método para bisagras del paper (sección 4.3, Fig. 7) se adapta a esa estructura. Se elige el plano vertical del brazo y se itera FABRIK en 2D, con forward, backward y los tres criterios de corte.
3) Dentro del plano aparece el problema del codo abajo, en el que FABRIK converge a la solución espejo que este brazo no puede hacer. Ajustar los ángulos al final o aplicar los límites en cada paso, por sí solos, no lo resuelven.
4) La técnica *pole* de Blender y Maya resuelve esta ambigüedad. En nuestro brazo se traduce en espejar el codo respecto de la recta hombro→objetivo cuando cae del lado prohibido; como el espejo conserva las distancias, no altera el funcionamiento de FABRIK.
5) Con restricciones en cada paso y la regla del codo (v3), la simulación resuelve el 99 % de los objetivos alcanzables partiendo de la pose cero, con 3.1 iteraciones en promedio.
6) El usuario solo indica el punto. La muñeca y la pinza las elige una búsqueda que empieza en su posición neutra y prueba otras de a 10°, primero en la pinza y después en la muñeca junto con la pinza. La búsqueda resuelve el 99.75 % de los objetivos de la prueba, contra el 84.95 % con la muñeca y la pinza siempre neutras.
7) Los valores numéricos de este informe salen de las medidas del brazo armado y de los límites del firmware. El método no depende de esas medidas, y antes del pasaje a C falta medir los topes reales de los servos.

Referencias

[1] A. Aristidou y J. Lasenby, «FABRIK: A fast, iterative solver for the Inverse Kinematics problem», *Graphical Models*, vol. 73, n.º 5, pp. 243-260, 2011, [En línea]. Disponible en: https://www.andreasaristidou.com/publications/papers/FABRIK.pdf

[2] Blender Foundation, «Inverse Kinematics Constraint — Blender Manual». [En línea]. Disponible en: https://docs.blender.org/manual/en/latest/animation/constraints/tracking/ik_solver.html

[3] Autodesk, «Rotate Plane IK solver — Maya Help». [En línea]. Disponible en: https://help.autodesk.com/cloudhelp/2017/ENU/Maya/files/GUID-9942FFB5-65C2-46E2-B5A3-297667A9FB5D.htm

[4] Autodesk, «poleVectorConstraint command — Maya Python Commands». [En línea]. Disponible en: https://help.autodesk.com/cloudhelp/2026/ENU/Maya-Tech-Docs/CommandsPython/poleVectorConstraint.html

[5] Grupo J5, «Informe Final — Brazo Robótico Seguro». Taller de Proyecto II, Facultad de Informática, UNLP, 2025.

[6] Epic Games, «FABRIK Animation Blueprint in Unreal Engine — Unreal Engine 5.8 Documentation». [En línea]. Disponible en: https://dev.epicgames.com/documentation/en-us/unreal-engine/fabrik-animation-blueprint-in-unreal-engine

[7] RootMotion, «Final IK — RootMotion». [En línea]. Disponible en: https://root-motion.com/final-ik.html

[8] MoveIt, «Kinematics Configuration — MoveIt Documentation». [En línea]. Disponible en: https://moveit.picknik.ai/main/doc/examples/kinematics_configuration/kinematics_configuration_tutorial.html